

Market Basket Analysis

Finding frequent itemsets in the IMDB Movies Dataset

Greta Zehnder

May 2026

Contents

1	Introduction	2
2	Dataset and Preprocessing	2
2.1	Dataset	2
2.2	Preprocessing	2
3	Exploratory Analysis and Support Threshold Selection	3
3.1	Actor Frequency Analysis	3
3.2	Support Threshold Selection	3
3.3	Transaction Encoding	3
4	Frequent Itemset Mining Algorithms	4
4.1	Apriori	4
4.2	PCY	4
4.3	SON	5
5	Results	5
5.1	Frequent Itemsets	5
5.2	Algorithm Comparison	6
6	Scalability Experiment	6
6.1	Experimental Setup	6
6.2	Results	7
6.3	Discussion	7
7	Conclusions	8
	Declaration of Academic Integrity	10

1 Introduction

Market basket analysis is a data mining technique aimed at discovering associations among items that frequently co-occur in transactions. Originally developed in the context of retail, where the goal is to identify products that customers tend to purchase together, the approach generalizes naturally to any domain where data can be represented as a collection of sets.

In this project, the technique is applied to a movie dataset, where each film is modeled as a basket and its main actors as items. The goal is to discover frequent co-acting patterns, i.e., groups of actors who appear together in multiple films.

Three algorithms are implemented and compared: Apriori, used as a baseline; PCY (Park–Chen–Yu), which reduces memory usage through hash-based candidate pruning; and SON (Savasere–Omicinski–Navathe), which adopts a partition-based strategy designed to scale in distributed environments. All three are evaluated for correctness and computational efficiency, with a particular focus on scalability to larger datasets.

2 Dataset and Preprocessing

2.1 Dataset

The dataset used in this project is the *IMDB Top 1000 Movies and TV Shows Dataset* [1], publicly available on Kaggle under the Public Domain license. It contains information about the top 1000 movies and TV shows ranked on IMDB, and includes 16 columns: poster link, title, release year, age certificate, runtime, genre, IMDB rating, overview, Metascore, director, four actor fields (**Star1**, **Star2**, **Star3**, **Star4**), number of votes, and gross revenue.

For the purposes of this project, only the four actor columns are relevant. Each row is treated as a transaction (basket), and the actors listed in **Star1**–**Star4** are treated as items.

2.2 Preprocessing

The preprocessing pipeline consists of the following steps:

1. **Column selection.** Only the four actor columns (**Star1**, **Star2**, **Star3**, **Star4**) are retained.
2. **Missing value removal.** Rows containing at least one missing value are dropped using `dropna()`. After this step, all 1000 rows are preserved, meaning no missing values were present in the actor columns.
3. **Transaction construction.** Each row is converted into a list of four actors, resulting in 1000 transactions.

The support-based filtering applied to reduce dimensionality is described in Section 3.

3 Exploratory Analysis and Support Threshold Selection

3.1 Actor Frequency Analysis

Before mining frequent itemsets, the frequency distribution of actors across all transactions is analyzed. This step serves three purposes: determining the number of distinct items, identifying the most frequently appearing actors, and informing the choice of a minimum support threshold.

The dataset contains 2709 distinct actors. Table 1 shows the top 10 most frequent actors.

Actor	Appearances
Robert De Niro	17
Tom Hanks	14
Al Pacino	13
Brad Pitt	12
Clint Eastwood	12
Christian Bale	11
Leonardo DiCaprio	11
Matt Damon	11
James Stewart	10
Michael Caine	9

Table 1: Top 10 most frequent actors in the dataset.

3.2 Support Threshold Selection

A minimum support threshold of 1% (at least 10 appearances) was initially considered. However, this proved too restrictive, yielding very few frequent actors and limiting the discovery of meaningful co-occurrence patterns.

The threshold was therefore lowered to 0.5% (at least 5 appearances). At this level, 79 actors are retained and 399 baskets contain at least one of them, providing a richer set of frequent itemsets while maintaining computational feasibility.

Filtering actors below the minimum support reduces the number of distinct items, which in turn lowers memory usage and speeds up frequent itemset mining, making the approach more scalable to larger datasets.

3.3 Transaction Encoding

The filtered transactions are encoded into a one-hot matrix using `TransactionEncoder` from the `mlxtend` library. A sparse representation is adopted to reduce memory usage. The resulting matrix has shape 399×79 (baskets $\times$ actors).

4 Frequent Itemset Mining Algorithms

4.1 Apriori

Apriori is the foundational algorithm for frequent itemset mining. It is based on the **antimonotonicity property**: if an itemset is infrequent, all of its supersets are infrequent as well. This property is used to prune the candidate space, avoiding the enumeration of all possible itemsets.

The algorithm proceeds in passes over the dataset. In the first pass, it counts the support of all individual items and retains those that meet the minimum support threshold. In each subsequent pass k , it generates candidate k -itemsets from the frequent $(k - 1)$ -itemsets found in the previous pass, counts their support, and prunes those below the threshold. The process continues until no new frequent itemsets are found.

In this project, Apriori is used as a baseline and is applied via the `apriori()` function from the `mlxtend` library, using the one-hot encoded sparse matrix described in Section 3. A minimum support of 0.5% is applied. The algorithm identifies 102 frequent itemsets in total, of which 21 are pairs. The top 10 frequent pairs are shown in the Results section.

4.2 PCY

PCY (Park–Chen–Yu) is a scalable extension of Apriori designed to reduce memory usage during the candidate generation phase. It introduces a extbfhash-based filtering step in the first pass over the data.

The algorithm proceeds in two passes:

1. **Pass 1.** For each transaction, individual item counts are updated as in Apriori. Additionally, all pairs of items are hashed into buckets, and bucket counts are incremented. At the end of this pass, a bitmap is constructed: a bucket is marked as frequent (bit = 1) if its count meets the minimum support threshold.
2. **Pass 2.** Only pairs where both items are frequent *and* whose hash falls in a frequent bucket are counted. This pruning significantly reduces the number of candidate pairs evaluated.

In this project, PCY is implemented from scratch in Python using a custom hash function. The hash table uses 2000 buckets. Table 2 summarizes the statistics from the two passes.

Statistic	Value
Number of baskets	399
Minimum count threshold	2
Distinct items after filter	79
Number of buckets	2000
Frequent buckets	23
Candidates counted in pass 2	69
Distinct pairs counted in pass 2	27
Frequent pairs found	21

Table 2: PCY algorithm statistics.

PCY reduces the number of pair evaluations in the second pass from 181 (naive counting) to 69, yielding a reduction factor of approximately $2.6\times$. The frequent pairs found are identical to those returned by Apriori, confirming the correctness of the implementation.

4.3 SON

The SON algorithm (Savasere–Omiecinski–Navathe) is a partition-based approach designed for scalable frequent itemset mining in distributed environments. Rather than scanning the entire dataset at once, it splits the data into partitions and mines each one independently, then performs a global verification pass.

The algorithm proceeds in two phases:

1. **Phase 1 (Local mining).** The dataset is divided into p partitions. For each partition, frequent itemsets are mined locally using a scaled minimum support threshold (proportional to the partition size). The union of all locally frequent itemsets forms the set of global candidates.
2. **Phase 2 (Global counting).** Each candidate is counted across the entire dataset. Only candidates meeting the global minimum support threshold are retained as frequent itemsets.

A key property of SON is that any globally frequent itemset must be locally frequent in at least one partition. This guarantees that no true frequent itemset is missed, while false positives are eliminated in the second pass.

In this project, SON is implemented from scratch in Python using 5 partitions. Table 3 summarizes the execution statistics.

Statistic	Value
Number of baskets	399
Global minimum count	2
Number of partitions	5
Candidates generated	139
Frequent pairs found	21

Table 3: SON algorithm statistics.

SON produces the same 21 frequent pairs as Apriori and PCY, confirming correctness. The higher number of candidates (139 vs. 69 for PCY) reflects the conservative nature of the local mining step, which over-generates candidates to ensure no frequent itemset is missed.

5 Results

5.1 Frequent Itemsets

Apriori identifies 102 frequent itemsets in total, the majority of which are single actors. Focusing on pairs, 21 frequent actor pairs are found. Table 4 shows the top 10 pairs sorted by support.

Actor pair	Support	Count
Rupert Grint, Daniel Radcliffe	0.015038	6
Rupert Grint, Emma Watson	0.012531	5
Daniel Radcliffe, Emma Watson	0.012531	5
Joe Pesci, Robert De Niro	0.010025	4
Al Pacino, Robert De Niro	0.007519	3
Al Pacino, Diane Keaton	0.007519	3
Toshirō Mifune, Tatsuya Nakadai	0.007519	3
Mark Ruffalo, Robert Downey Jr.	0.007519	3
Scarlett Johansson, Chris Evans	0.007519	3
Michael Caine, Christian Bale	0.007519	3

Table 4: Top 10 frequent actor pairs found by Apriori.

The most frequent co-acting pattern is (Rupert Grint, Daniel Radcliffe) with a count of 6, reflecting their repeated collaboration in the Harry Potter series. The top three pairs all involve Harry Potter cast members, which is consistent with the franchise’s large number of entries in the IMDB Top 1000.

5.2 Algorithm Comparison

All three algorithms (Apriori, PCY, and SON) return the same set of 21 frequent pairs, with identical counts. Table 5 summarizes the comparison.

	Apriori	PCY	SON
Frequent pairs found	21	21	21
Same set as Apriori	—	Yes	Yes
Count mismatches	—	0	0
Candidates evaluated	—	69	139

Table 5: Comparison of the three algorithms on the filtered dataset.

The results confirm the correctness of both custom implementations. PCY and SON produce results identical to the Apriori baseline, with no discrepancies in either the set of frequent pairs or their support counts.

6 Scalability Experiment

6.1 Experimental Setup

To evaluate scalability beyond the original dataset size, the filtered transaction list (399 baskets) is replicated by a factor $k \in \{1, 5, 10\}$, yielding datasets of 399, 1995, and 3990 baskets respectively. This approach preserves the original data distribution while increasing the number of transactions. The runtime of each algorithm is measured using Python’s `time` module.

6.2 Results

Table 6 reports the runtime of each algorithm for each dataset size.

k	Baskets	Apriori (s)	PCY (s)	SON (s)
1	399	0.0249	0.0012	0.0030
5	1995	0.0494	0.0049	0.0148
10	3990	0.0863	0.0092	0.0192

Table 6: Runtime (in seconds) of each algorithm as dataset size increases.

Figure 1 shows the runtime growth as a function of the number of baskets.

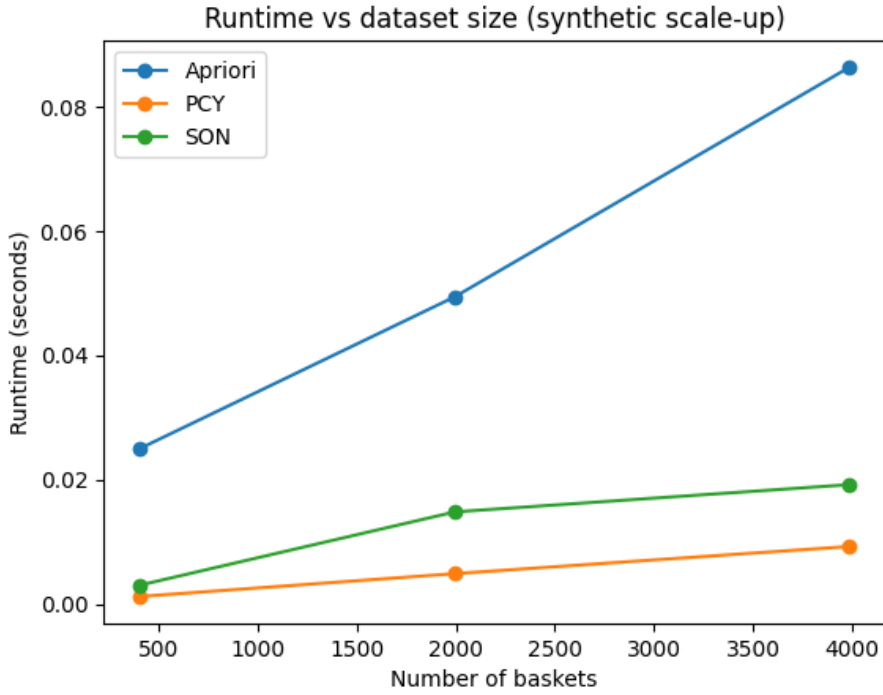


Figure 1: Runtime vs dataset size (synthetic scale-up).

6.3 Discussion

PCY is consistently the fastest algorithm across all dataset sizes, outperforming Apriori by approximately one order of magnitude. Its runtime grows slowly as the dataset size increases, thanks to the hash-based pruning that limits the number of candidates evaluated in the second pass.

SON is slower than PCY but significantly faster than Apriori. Its runtime grows more steeply than PCY due to the global counting phase in pass 2, which scans all transactions for each candidate. However, its partition-based design makes it well-suited for distributed environments, where each partition can be processed independently in parallel.

Apriori shows the steepest growth, reflecting its higher computational cost as the number of transactions increases.

It should be noted that this experiment uses a synthetic scale-up by replication, which preserves the original data distribution. On a real larger dataset with more distinct actors and sparser co-occurrences, the differences between algorithms could be more pronounced.

7 Conclusions

This project applied market basket analysis to the IMDB Top 1000 Movies and TV Shows Dataset, modeling movies as transactions and actors as items. The goal was to discover frequent co-acting patterns using three algorithms: Apriori, PCY, and SON.

After preprocessing and support-based filtering (threshold: 0.5%), the dataset was reduced to 399 baskets and 79 distinct actors. Frequent itemset mining identified 102 frequent itemsets, of which 21 are pairs. The most frequent co-acting pattern is (Rupert Grint, Daniel Radcliffe), reflecting their repeated collaboration in the Harry Potter series.

All three algorithms produced identical results, confirming the correctness of the PCY and SON implementations. In terms of efficiency, PCY proved the fastest, reducing candidate pair evaluations by a factor of approximately $2.6\times$ compared to naive counting. SON generated more candidates than PCY due to its conservative local mining strategy, but its partition-based design makes it inherently more suitable for distributed and large-scale settings.

The scalability experiment showed that both PCY and SON scale significantly better than Apriori as dataset size increases. A limitation of this experiment is that the scale-up was performed by replicating the original transactions, which preserves the data distribution. On a real larger dataset — with more distinct actors and sparser co-occurrences — the performance differences between algorithms could be more pronounced.

Future work could explore association rule mining on top of the frequent itemsets found, or apply the same pipeline to a larger and more diverse movie dataset to better stress-test the scalability of the proposed solution.

References

- [1] Harshit Shankhdhar, *IMDB Movies Dataset*, Kaggle.
<https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows>

Declaration of Academic Integrity

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.